Stippler

Project Report

Barnabas Tanczos

15 May 2026

Abstract

Stippling is an artistic technique that represents tone and texture through the placement of dots. This report presents an implementation and comparison of four different stippling algorithms in a C++ application: naive rejection sampling, weighted centroidal Voronoi tessellation (CVT), adaptive incremental stippling (AIS), and recursive Wang tile stippling. The techniques are evaluated based on their performance, spectral qualities, and a proposed perceptual study.

1 Introduction

Stippling is a core technique of non-photorealistic rendering that involves the placement of discrete dots, typically black on white, to convey the form and tone of the image through their arrangement and density (see Figure 1). The traditional art form has a long history across various mediums, and there has been a sustained interest in replicating it using computers. As a result, a number of different techniques have been developed in the past decades for converting images into visually pleasing stipples [1]. In some cases, research in this area has created entirely new ways of artistic expression.

Producing a high-quality stippled image algorithmically is challenging. On one hand, dots placed with high randomness will create clusters and

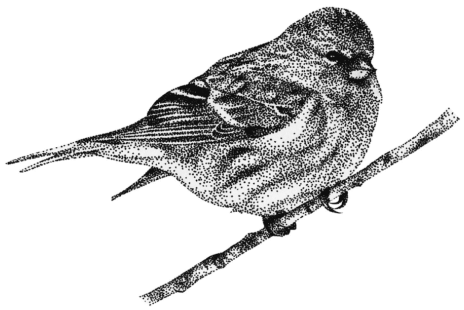


Figure 1: An example of hand-drawn stippling [2].

leave gaps, which results in an uneven and visually unappealing result. At the other end, high regularity is also undesirable as it feels artificial and loses the organic quality of hand-drawn dots. The ideal between these – local randomness but no large-scale fluctuations – is known as blue noise. Thus, the algorithmic strategies for stippling and generating blue noise greatly overlap, with the important distinction that stippling requires the distribution to adapt to variations in image density rather than remaining uniform over the entire surface.

Several broad categories of methods have been proposed to satisfy these requirements, each with different trade-offs regarding quality, responsiveness and speed. This project implements four representative methods within a single C++ application: naive rejection sampling, weighted centroidal Voronoi tessellation (CVT) [3], adaptive incremental stippling (AIS) [4], and recursive Wang tile stippling [5]. The aim is to provide a cohesive quantitative and qualitative comparison of these algorithms, in terms of performance, blue noise spectral properties (using the power spectrum, radially averaged power, and anisotropy), stippling characteristics and usability, as well as a proposed perceptual study.

2 Related Work

Stippling has been the subject of extensive research. Martín et al.[1] provide a comprehensive overview and classification of these methods, and Yan et al.[6] contextualize them within the larger field of blue noise generation. This project focuses on three major approaches: relaxation, incremental, and tile-based.

The first category aims to evenly distribute stipples and avoid any closeness. For this purpose, Secord [3] proposed using Lloyd's method to iteratively move the dots of an initial point distribution towards the centroids of their Voronoi diagram. This relaxation process converges to a centroidal Voronoi diagram (CVD) weighted by the tone of the image. The technique produces highly regular distributions, a concern that was addressed by Balzer et al.[7] by introducing a capacity constraint for each Voronoi region. Due to their iterative nature, these algorithms are relatively slow, however various optimizations have been implemented such as in de Goes et al.[8].

To improve the performance and blue noise qualities, incremental methods based on Poisson disk distributions have also been suggested. A foundational method for sampling a Poisson disk distribution for arbitrary dimensions is given by Bridson [9]. The algorithm generates candidates in an annulus around existing points and accepts those that satisfy the minimum

distance constraint. The stippling technique of Ascencio-Lopez et al.[4] follows a similar approach, but takes into account the local image density to adjust the radius of each disk.

Finally, in order to achieve an even faster runtime, algorithms may sample from a pre-computed set of points. This is the approach of Kopf et al.[5], who adjust an initial distribution to form Wang tiles that can be placed adjacent to each other seamlessly. Moreover, recursion rules are established to allow arbitrary point density in any region. There exist similar approaches with different tiling strategies, such as Ostromoukhov's [10] method based on polyominoes.

Further methods exist that refine the stippling process or enable new ways of expression: animated stippling of 3D models [11], different dot shapes [12], emulation of specific artists' styles [13], and awareness of linear features in the image [14], among others.

3 Implementation

The stippling algorithms were implemented within a C++ application with a command line interface using the SDL2 library for graphics. The application converts an input image to a stippled version or generates a uniform dot distribution using one of the four available algorithms: naive rejection, CVT, AIS, Wang tiles. The first of these, which simply generates random points and accepts them with probability equal to the image intensity, is used as a baseline for comparison. The program supports a number of typical additional features: colour inversion, saving to a file, removing points above a luminosity threshold, and modes for single pixel dots, fixed radius dots, and dots scaled by intensity.

3.1 Centroidal Voronoi tessellation (CVT)

The stippling algorithm proposed by Secord [3] uses a centroidal Voronoi diagram (CVD) weighted by image density to produce a well-distributed set of points. As a relaxation method, it begins with an initial distribution and iteratively improves it until it converges.

A Voronoi diagram partitions the plane into regions based on a set of generating points, where each region contains the points closer to its generator than any other (see Figure 2). Formally, the Voronoi region V_i corresponding to generator $\mathbf{x}_i$ in an area A is

$$V_i = \{\mathbf{x} \in A : |\mathbf{x} - \mathbf{x}_i| \leq |\mathbf{x} - \mathbf{x}_j| \ \forall i \neq j\}.$$

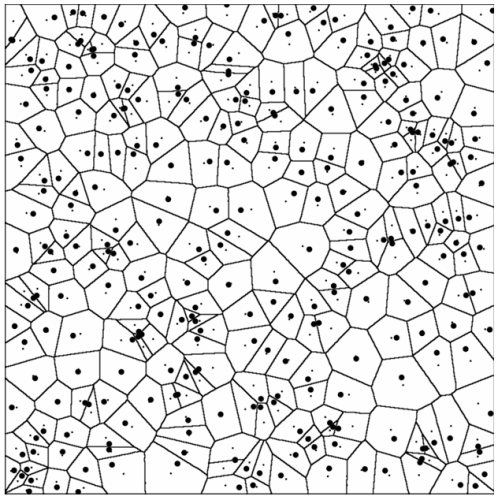


Figure 2: Voronoi diagram with generators (large dots) and centroids (small dots) [3].

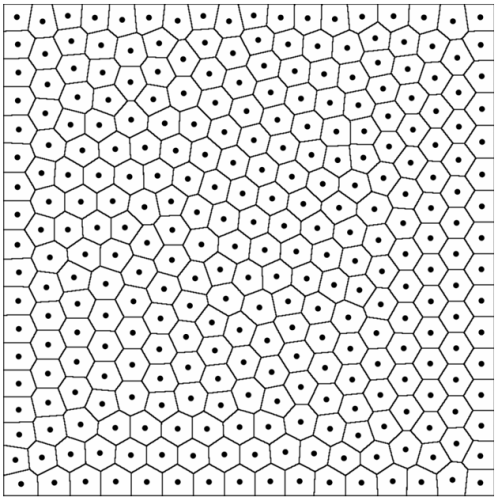


Figure 3: Centroidal Voronoi diagram (dots are both generators and centroids) [3].

The centroid of a Voronoi region is defined as

$$\mathbf{c}_i = \frac{\int_{V_i} \mathbf{x}\rho(\mathbf{x})dA}{\int_{V_i} \rho(\mathbf{x})dA}$$

where ρ is a density function. It is the centre of mass of the region. A CVD is a Voronoi diagram in which every generator is also the centroid of its region (see Figure 3). For stippling, the density can be set to the intensity of the image at the pixel, and the integral is approximated by a sum of the pixels in the area. Constructing this CVD results in an evenly spaced stippling that follows the image tone.

Lloyd's method is an iterative algorithm to compute a CVD from any set of n generating points (see Algorithm 1). Starting from an initial point set, given by the existing rejection sampling algorithm for a fast approximation, it alternates between computing the Voronoi diagram and moving each generator to the centroid of the region.

The most computationally expensive step is the nearest neighbour query, which is performed on each pixel every iteration to find its Voronoi region. A variety of algorithms exist for this problem. The implementation in this project uses a simple uniform spatial grid, where each cell stores a list of generators that fall within it. The grid cell size is set to $2\sqrt{WH/n}$, twice the expected average spacing. The neighbour search expands outwards from the

Algorithm 1: Weighted CVT stippling

```
points ← RejectionSample(image, n)
repeat 16 times
    mass[], moments[] ← empty
    foreach (x, y) in image do
        i ← NearestNeighbour((x, y), points)
        mass[i] += ρ(x, y)
        moments[i] += ρ(x, y) · (x, y)
    for i ← 0 to n do
        points[i] ← moments[i]/mass[i]
return points
```

pixel's grid cell in rings of increasing radius, stopping as soon as a nearest point is found that is guaranteed to be closer than any unsearched cell. Because points are dense in many cases, this is relatively efficient.

The paper suggests assessing convergence using average displacement of generating points; however such a threshold was found to be difficult to define across all image sizes. Therefore, the relaxation process stops after a fixed number of iterations.

3.2 Adaptive incremental stippling (AIS)

The AIS algorithm, proposed by Ascencio-Lopez et al.[4], incrementally generates a Poisson disk distribution adapted to the tone of an image by sampling in the neighbourhood of existing points and adjusting the radius of disks to match the local density.

Each stipple is modelled as a disk with a centre and radius. Unlike a regular Poisson disk distribution, the radius is not fixed. Instead, all disks have the same local density, defined as the sum of intensity values over the pixels within the disk. This density is approximately equal to K, a user-defined parameter that allows control over how densely dots are packed and therefore the number of dots. The total density T is the sum of all pixel intensities in the entire image.

The algorithm (see Algorithm 2) starts with a queue that contains a disk with a random position. All disks begin with an estimated radius

$$r_1 = \sqrt{WH/(T/K)}$$

which is the expected area (width W by height H) over the expected number of dots. This radius is then iteratively adjusted by the ratio $\sqrt{K/l}$, where l is the current local density. After a few iterations, this ratio converges to approximately 1. If this does not occur after six iterations, the algorithm always chooses to underestimate to avoid obvious empty circles in the final image.

Algorithm 2: Adaptive incremental stippling

$queue \leftarrow \varnothing$
$diskBuffer[] \leftarrow$ empty

$D \leftarrow$ `Disk(`$(x_{\text{random}}, y_{\text{random}})$, r_1`)`
`ChangeRadiusBasedOnLocalDensity(`D, K, *image*`)`
`FillDisk(`*diskBuffer*, D`)`
$queue \leftarrow D$
$points \leftarrow (x_D, y_D)$

while $queue \neq \varnothing$ **do**
  $Q \leftarrow queue$
  while `AvailableRange(`Q`)` > 0 **do**
    $\alpha \leftarrow$ `RandomAngle(AvailableRange(`Q`))`
    $D \leftarrow$ `Disk(`$(x_Q + (r_Q + r_1)\cos\alpha, y_Q + (r_Q + r_1)\sin\alpha)$, r_1`)`
    `ChangeRadiusBasedOnLocalDensity(`D, Q, K, *image*`)`

    if `NoOverlap(`D, *diskBuffer*`)` **then**
      `FillDisk(`D, *diskBuffer*`)`
      $queue \leftarrow D$
      $points \leftarrow (x_D, y_D)$

    `SubtractAvailableRange(`Q, D`)`

return *points*

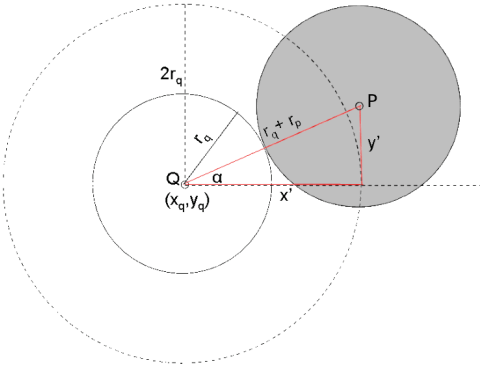


Figure 4: Distance calculation in AIS algorithm. [4]

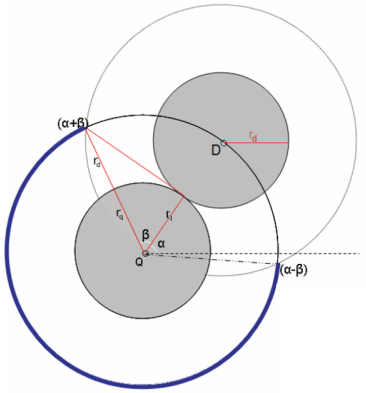


Figure 5: Angle calculation in AIS algorithm [4].

While the queue is non-empty, a new disk D is generated adjacent to the existing disk Q at a random angle α. The centre of this disk is calculated as

$$(x_P, y_P) = (x_Q + (r_Q + r_P)\cos\alpha, y_Q + (r_Q + r_P)\sin\alpha)$$

and is updated as the radius is adjusted to match the local density (see Figure 4). A candidate disk is then accepted if it does not overlap with any other disk to maintain the Poisson disk condition. This check is done using a buffer that keeps track of filled pixels, which avoids issues with varying radii but can likely be further optimized.

Regardless of whether the disk is accepted, the angle where it was placed is marked as unavailable on the original disk. This avoids any new attempts in regions where it is certain that there would be an overlap. Every disk initially has the full circle $[0, 2\pi)$ available. The arc $[\alpha - \beta, \alpha + \beta]$ is removed, where

$$\beta = \arccos\left(\frac{r_Q}{r_Q + r_D}\right)$$

based on the radii of the disks (see Figure 5). Once there are no disks left with available angles, the algorithm is finished.

3.3 Recursive Wang tile stippling

The recursive Wang tile stippling algorithm by Kopf et al.[5] samples deterministically from a pre-computed set of blue noise distributions. This method is near instant because it only adapts an input tile set to the image density rather than generating a new well-spaced point set.

Wang tiles are square tiles with a colour conceptually assigned to each edge. A set of tiles can be arranged to cover the plane by matching adjacent edge colours, without rotation or reflection. A sufficiently large set of Wang tiles will tile the plane aperiodically, so the arrangement never repeats.

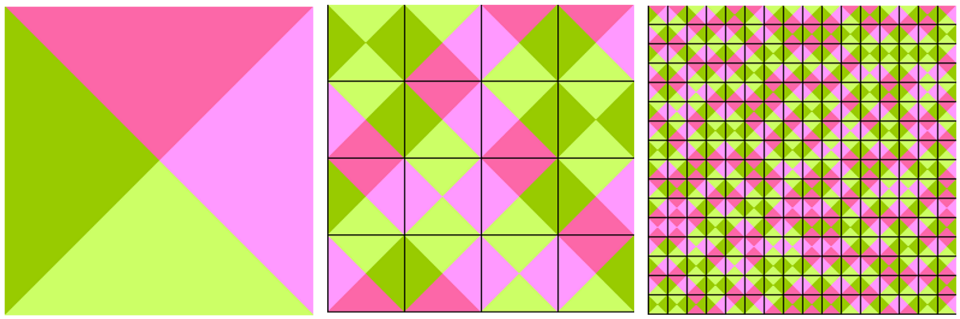


Figure 6: Recursive subdivision of Wang tiles following colour rules [5].

The authors construct a tile set of blue noise point distributions such that placing two tiles with matching edge colours adjacent to each other preserves their blue noise properties. Each tile additionally obeys a subdivision rule so that it can be decomposed into a grid of $k \times k$ scaled down Wang tiles according to a lookup table. Importantly, the points in the parent tile are a subset of the points in the subdivision, such that the blue noise properties are also preserved after any level of refinement (see Figure 6). The tile set used in this implementation was provided by the original authors and loaded from a binary file, as generating a new one is a complex problem outside the scope of this project.

The algorithm itself (see Algorithm 3) first samples from the root tile. Its points are iterated in a fixed order, which produces a consistent deterministic pattern, and each point is accepted based on whether the image intensity at that pixel falls below a threshold proportional to its rank within the tile. Specifically, point i is accepted if

$$\rho(\mathbf{x}_i) < i/s$$

where ρ is the density and s is a parameter provided by the user controlling the maximum number of points.

Then, the algorithm recurses into subtiles. At each level, an additional set of subpoints – belonging to the subtiles but not the parent tile – are similarly tested, except that the probabilities and number of tests are scaled down to match the size of the subtile, which has area $1/k^{2 \cdot level}$. Recursion continues

Algorithm 3: Recursive Wang tile stippling

for $i \leftarrow 0$ **to** $\min(|tile_0.points|,\ s)$ **do**
  $p \leftarrow tile_0.points[i]$
  if $image[p] < i\ /\ s$ **then**
    $points \leftarrow p$
`StippleRecursive(`*image, tile*$_0$*, (0, 0), 0*`)`
return *points*

Function `StippleRecursive(`*image, tile, corner, level*`):`
  $tileSize \leftarrow 1\ /\ k^{level}$
  $target \leftarrow s\ /\ k^{2 \cdot level} - |tile.points|$
  for $i \leftarrow 0$ **to** $\min(|tile.subPoints|,\ target)$ **do**
    $p \leftarrow corner + tile.subPoints[i] \cdot tileSize$
    if $image[p] < (i + |tile.points|) \cdot k^{2 \cdot level}/s$ **then**
      $points \leftarrow p$
  if $target > |tile.subPoints|$ **then**
    for $x, y \leftarrow 0$ **to** k **do**
      `StippleRecursive(`*image, tiles*$[tile.subtiles[x][y]]$*,*
       corner $+ (x, y) \cdot tileSize\ /\ k,$ *level* $+ 1$`)`

only if the target density for the region exceeds the number of points already generated. Therefore, lighter regions terminate early and darker regions are refined deeply.

4 Evaluation

Figures 7 and 8 show the output of all four algorithms on two test images, approximately matched by point count. The Grace Kelly image was rendered with variable dot size proportional to the intensity, while the mask was rendered with single-pixel dots.



(a) Original image.



(b) Rejection (0.2 s).



(c) CVT (28.0 s).



(d) AIS (10.4 s).



(e) Wang tiles (0.2 s).

Figure 7: An image stippled using the four algorithms with variable dot size.



(a) Original image.



(b) Rejection (0.1 s).



(c) CVT (12.0 s).



(d) AIS (2.0 s).



(e) Wang tiles (0.1 s).

Figure 8: An image stippled using the four algorithms with pixel dot size.

4.1 Performance

Table 1 summarises the runtimes recorded for each algorithm on both test images. The timings include only the stipple generation step and not the rendering pass, which is identical across algorithms. The results are as expected: Wang tile stippling and rejection are near-instant, CVT is the slowest by a significant margin and AIS is between the two.

Image	Rejection	CVT	AIS	Wang tiles
Grace Kelly	0.2 s	28.0 s	10.4 s	0.2 s
Mask	0.1 s	12.0 s	2.0 s	0.1 s

Table 1: Runtimes for each algorithm on each test image.

Rejection sampling accepts each random candidate with a certain probability, so the runtime depends on the number of points and acceptance rate. In the average case, the acceptance rate is a constant, so the algorithm is $O(n)$ with no storage overhead.

CVT performs I iterations of relaxation, where each one visits every pixel and performs a neighbourhood search. Using spatial grid acceleration, the neighbourhood query is usually small but varies strongly, while with other methods a more consistent $\log n$ can be achieved. The overall complexity is thus around $O(IWH)$.

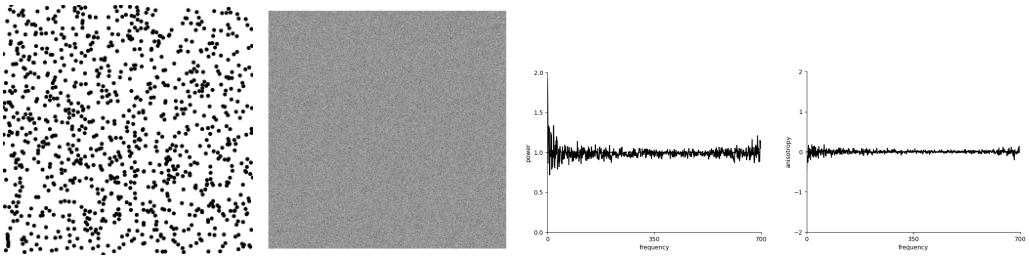
AIS has complexity $O(n)$ in the number of output points, but the operations performed still depend on the image resolution unless not every pixel is sampled. The runtime is significantly improved by limiting how large disks can grow in high-luminosity areas.

Finally, Wang tile stippling is also $O(n)$, as it operates on a sequence of predetermined points. Unlike AIS, it samples only a single pixel (or four using bilinear interpolation) per point, accounting for its high speed.

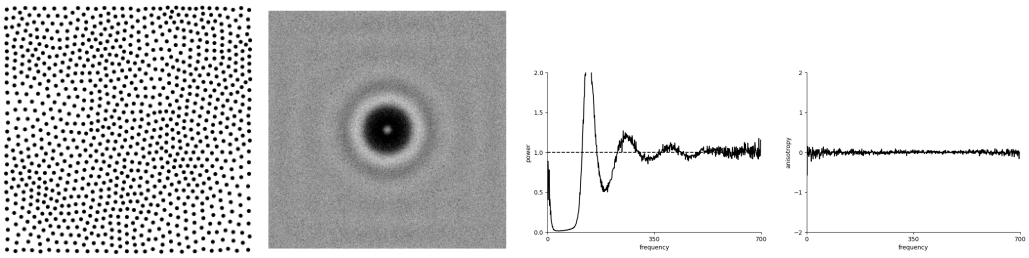
4.2 Spectral analysis

Spectral analysis is an important tool for studying the frequency characteristics of blue noise distributions, as described by Lagae and Dutré [15]. For these tests, the algorithms were run on an image with uniform density (see Figure 9).

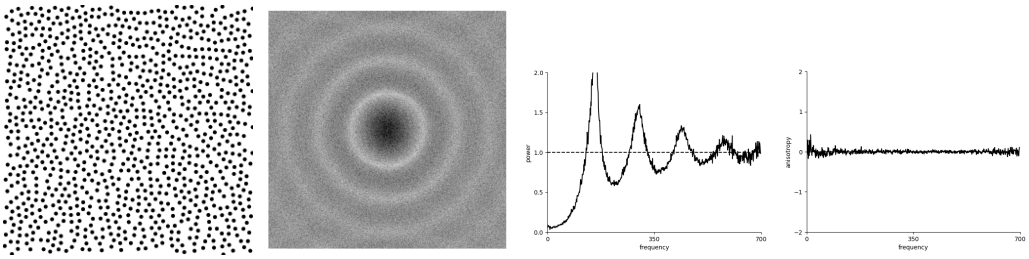
The *power spectrum* is computed as the squared magnitude of the 2D discrete Fourier transform of the image, normalized by point count. An ideal blue noise distribution produces a dark circle at the centre, indicating low



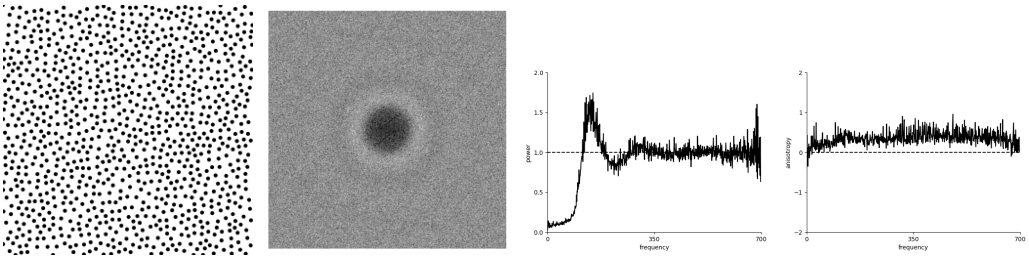
(a) Rejection.



(b) CVT.



(c) AIS.



(d) Wang tiles.

Figure 9: For each algorithm: points generated from a uniform density, the power spectrum, a graph of the radial average power, and a graph of anisotropy.

energy at low frequencies as there is no large-scale clustering, surrounded by brighter rings at even intervals, indicating the minimum distance between points. The *radial average power* provides a one-dimensional view of the same information by averaging the power spectrum over concentric rings of increasing radius. The *anisotropy*, meanwhile, is the variance of the power spectrum on each ring divided by the squared radial average power. It measures the radial symmetry of the distribution. Blue noise is ideally isotropic, meaning that it has no preferred direction, and therefore should have an anisotropy of zero.

As expected, rejection sampling produces a flat power spectrum with no meaningful patterns. Since every point has an equal probability of being selected on a uniform density, it produces white noise. CVT produces the most well-defined centre region, since the distribution is the most regular and evenly spaced. It strongly avoids any clusters, which results in a very uniform image but may not always be desirable for a more natural stippling. AIS strictly enforces a minimum distance between points, resulting in a prominent secondary rings, though the incremental approach does result in some large-scale chain patterns. Lastly, Wang tiles have weaker blue-noise characteristics, particularly regarding anisotropy, because the tiles were designed in levels of subdivision. If only a portion of the last level is included based on the required number of points, some points will be missing from the intended distribution, and thus the result will be more irregular.

4.3 Proposed perceptual study

A perceptual evaluation would complement the quantitative measurements by assessing how viewers subjectively perceive the quality of the stippled images. While spectral analysis captures important aspects of the underlying distributions, a user study will provide a holistic insight into whether the image appears visually pleasing, natural, and faithful to the original image.

The proposed study would compare the four implemented algorithms using a repeated measures design. A suitable set of participants would consist of 20–30 students with normal vision. Participants would be shown stippled images generated from a shared set of input images with varied characteristics, such as portraits, landscapes, and images with high contrast or fine detail. The generated renderings would be approximately matched by the number of stipples. Each participant would evaluate the images according to several criteria using a Likert scale, including:

- **visual quality**: overall aesthetic appeal;
- **tone**: how accurately shading and contrast are preserved;

- **details**: how well fine details remain recognizable;
- **naturalness**: whether the image appears hand-drawn or synthetic;
- **uniformity**: absence of clumping or gaps.

To reduce ordering effects, the presentation of images would be randomized for each participant. They would also not know which algorithm generated which image.

In addition to the rating scales, participants could also complete pairwise comparisons, where two stippled versions of the same image are shown side by side and participants select the one they prefer. This is informative about the overall impression of each image. Participants may find it easier to compare images rather than judge them on their own.

Statistical analysis could be performed on the results using repeated measures ANOVA or the Friedman test. The results would help determine whether the blue noise metrics align with human perception of quality. In the case of stippling, a qualitative evaluation is particularly important because the ultimate practical usefulness of a stippling algorithm is largely dependent on how aesthetically pleasing its results are to viewers.

5 Future work

Several extensions and improvements could be explored beyond the scope of this project. Firstly, all of the algorithms have opportunities for optimization. The nearest neighbour search for CVT could likely be improved by more advanced algorithms, such as k-d trees or quadtrees, or by using an alternative to Lloyd's method altogether [16]. Similarly, the overlap check for AIS remains unoptimized. A GPU implementation would further help performance. It is important to note, however, that small runtime improvements have little practical use for stippling unless they are so fast that they may be used in real-time animation.

Further research could be incorporated to extend and improve certain properties of the algorithms. For instance, Balzer et al.[7] addressed the regularity artifacts of the CVT algorithm by introducing a capacity constraint. Similarly, Mould [14] and Li and Mould [17] proposed methods to better emphasize edges and important points of contrast, respectively, which could be integrated into the AIS algorithm. Naturally, there are also a variety of more modern alternatives to these algorithms that perform better in many aspects, such as Linde-Buzo-Gray stippling [18].

Some additional features could also improve the usability and breadth of the stippling program: support for more input formats, SVG output, contrast

enhancement, or user interface to adjust parameters. These would render the application more practical for creating art beyond a simple demonstration.

6 Conclusion

This project implemented four different approaches to stippling within a C++ framework: naive rejection sampling, weighted centroidal Voronoi tessellation, adaptive incremental stippling, and recursive Wang tile stippling. These methods represent major categories of stippling and blue noise generation techniques. The results demonstrate the trade-offs between image quality, blue noise properties, and performance: CVT generates very regular distributions at a high computational cost; Wang tiles are extremely fast but produce a weaker spectral quality; and AIS provides a compromise between quality and performance, with a more natural appearance that may sometimes be preferable.

Implementing these algorithms provided significant practical insight into sampling theory and the considerations in non-photorealistic rendering. Overall, the evaluation of stippling techniques highlights the importance of considering theoretical correctness and quantitative measurements alongside user perception, as algorithmic optimality may not always translate to aesthetic preference for humans.

References

[1] Domingo Martín, Germán Arroyo, Alejandro Rodríguez Aguilera, and Tobias Isenberg. A survey of digital stippling. *Computers & Graphics*, 67:24–44, 2017.

[2] Arvitalya. Redpoll illustration, 2017.

[3] Adrian Secord. Weighted Voronoi stippling. In *Proceedings of the 2nd International Symposium on Non-Photorealistic Animation and Rendering (NPAR)*, pages 37–43, New York, 2002. ACM.

[4] Ignacio Ascencio-Lopez, Oscar Meruvia-Pastor, and Hugo Hidalgo-Silva. Adaptive incremental stippling using the Poisson-disk distribution. *Journal of Graphics, GPU, and Game Tools*, 15(1):29–47, 2010.

[5] Johannes Kopf, Daniel Cohen-Or, Oliver Deussen, and Dani Lischinski. Recursive Wang tiles for real-time blue noise. *ACM Transactions on Graphics*, 25(3):509–518, 2006.

[6] Dong-Ming Yan, Jian-Wei Guo, Bin Wang, Xiao-Peng Zhang, and Peter Wonka. A survey of blue-noise sampling and its applications. *Journal of Computer Science and Technology*, 30(3):439–452, 2015.

[7] Michael Balzer, Thomas Schlömer, and Oliver Deussen. Capacity-constrained point distributions: A variant of Lloyd's method. *ACM Transactions on Graphics*, 28(3):86:1–86:8, 2009.

[8] Fernando de Goes, Katherine Breeden, Victor Ostromoukhov, and Mathieu Desbrun. Blue noise through optimal transport. *ACM Transactions on Graphics*, 31(6):171:1–171:11, 2012.

[9] Robert Bridson. Fast poisson disk sampling in arbitrary dimensions. In *ACM SIGGRAPH 2007 Sketches*, pages 22:1–22:5, 2007.

[10] Victor Ostromoukhov. Sampling with polyominoes. *ACM Transactions on Graphics*, 26(3):78:1–78:6, 2007.

[11] Oscar E. Meruvia Pastor, Bernd Freudenberg, and Thomas Strothotte. Real-time animated stippling. *IEEE Computer Graphics and Applications*, 23(4):62–68, 2003.

[12] Stefan Hiller, Holger Hellwig, and Oliver Deussen. Beyond stippling – methods for distributing objects on the plane. *Computer Graphics Forum*, 22(3):515–522, 2003.

[13] David Martín, Gabriel Arroyo, María V. Luzón, and Tobias Isenberg. Example based stippling using a scale-dependent grayscale process. In *Proceedings of the 8th International Symposium on Non-Photorealistic Animation and Rendering (NPAR)*, pages 51–61, New York, 2010. ACM.

[14] David Mould. Stipple placement using distance in a weighted graph. In *Proceedings of Computational Aesthetics in Graphics, Visualization, and Imaging (CAe)*, pages 45–52, Goslar, Germany, 2007. Eurographics Association.

[15] Ares Lagae and Philip Dutré. A comparison of methods for generating Poisson disk distributions. *Computer Graphics Forum*, 27(1):114–129, March 2008.

[16] James C. Hateley, Huayi Wei, and Long Chen. Fast methods for computing centroidal voronoi tessellations. *Journal of Scientific Computing*, 63(1):185–212, April 2015.

[17] Hao Li and David Mould. Structure-preserving stippling by priority based error diffusion. In *Proceedings of Graphics Interface*, pages 127–134, Waterloo, Canada, 2011. CHCCS.

[18] Oliver Deussen, Marc Spicker, and Qian Zheng. Weighted linde-buzo-gray stippling. *ACM Transactions on Graphics*, 36(6), November 2017.